

SchoolPay - Système de Gestion des Paiements Scolaires

Application web moderne pour la gestion automatisée des paiements des frais scolaires
Cas pilote : Complexe Scolaire Saint Vincent de Paul, Kinshasa

Table des matières

- Contexte
- Architecture Technique
- Équipe & Rôles
- Installation & Configuration
- Structure du Projet
- API Endpoints
- Sécurité
- Déploiement
- Roadmap

Contexte

SchoolPay modernise la gestion financière des établissements scolaires en remplaçant les processus manuels (registres papier, reçus écrits à la main) par une solution numérique complète.

Problématiques résolues

- ❌ Lenteur des enregistrements manuels
- ❌ Risques d'erreurs et de pertes de données
- ❌ Manque de traçabilité et de transparence
- ❌ Difficulté de suivi des impayés
- ❌ Génération laborieuse des rapports financiers

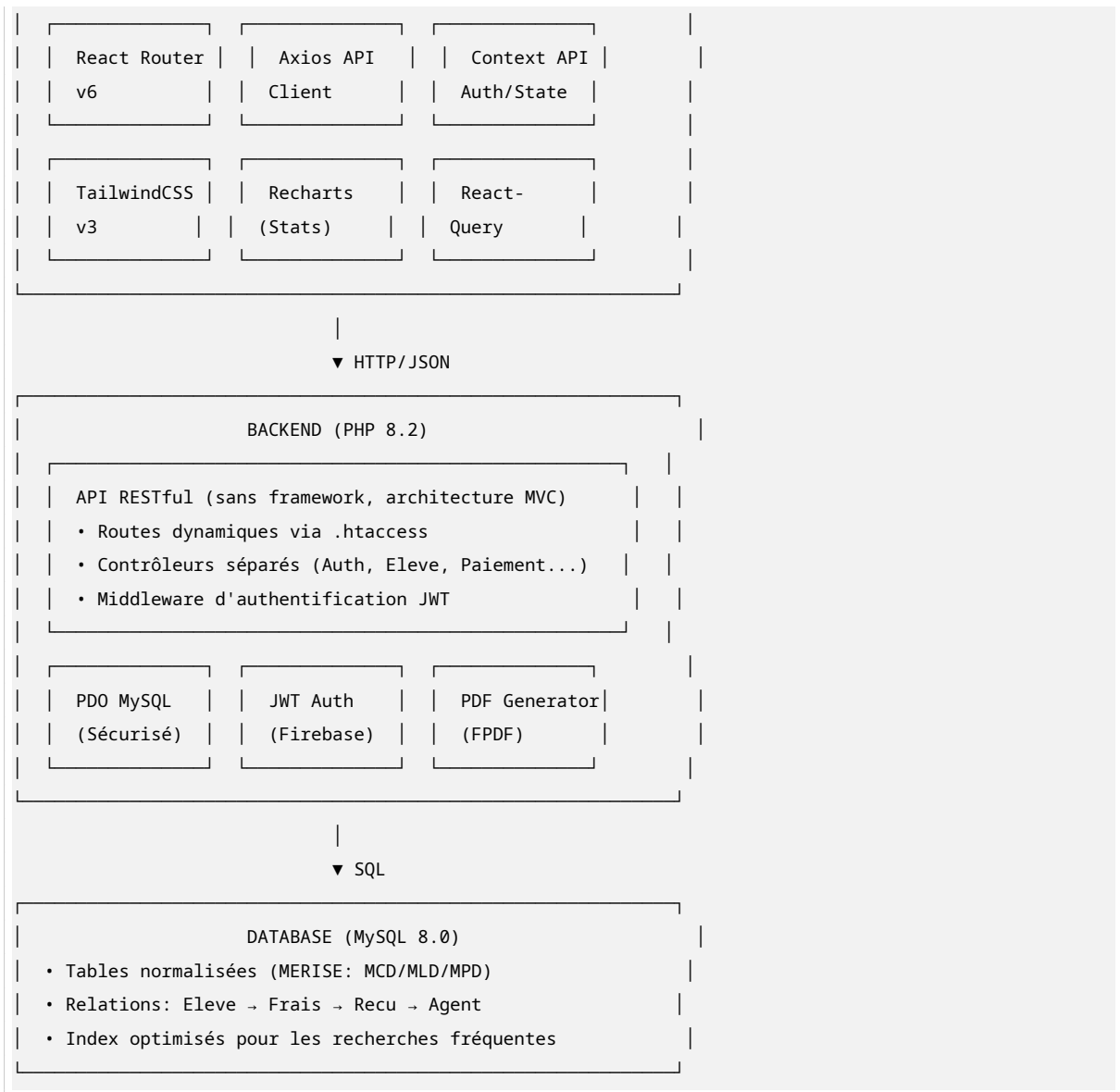
Solutions apportées

- ✅ Interface React moderne et responsive
- ✅ API PHP RESTful sécurisée
- ✅ Génération automatique de reçus PDF
- ✅ Paiement via mobile money (Orange Money, M-Pesa, Airtel Money)
- ✅ Tableau de bord analytique en temps réel
- ✅ Multi-école (scalable pour 4+ établissements)

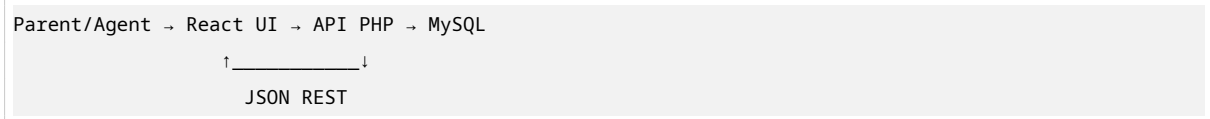
Architecture Technique

Stack Complet





Flux de données



👥 Équipe & Rôles

Membre	Rôle Principal	Responsabilités	Période
Ben Nguvulu	Lead Frontend	Architecture React, UI/UX, Composants réutilisables	Semaine 1-1.5
Tonnerre Kalomb	Frontend Dev	Interfaces paiement, Dashboard, Intégration API	Semaine 1-1.5

Table 1 – continued

Membre	Rôle Principal	Responsabilités	Période
Vital Mushale	Lead Backend	API PHP, Base de données, Sécurité, Logic métier	Semaine 2-3
Rossy Wannose	Fullstack Dev	Connexion front/back, Tests, Documentation	Semaine 1-3
Greg Faso	Infrastructure	Internet, Électricité, Déploiement local	Continu

Planning de développement (24 Mars - 20 Mai 2026)

```

title SchoolPay - Chronogramme
dateFormat YYYY-MM-DD
section Phase 1
Frontend (React)      :a1, 2026-03-24, 10d
section Phase 2
Backend (PHP/API)     :a2, 2026-04-06, 14d
section Phase 3
Base de données       :a3, 2026-04-20, 14d
section Phase 4
Tests & Intégration   :a4, 2026-05-04, 16d
section Déploiement
Lancement 4 écoles    :a5, 2026-05-10, 10d

```

Installation & Configuration

Prérequis

- **Node.js** $\geq 18.0.0$
- **PHP** ≥ 8.2 avec extensions: pdo_mysql, json, mbstring
- **MySQL** ≥ 8.0 ou **MariaDB** ≥ 10.6
- **Composer** (pour les dépendances PHP)
- **Git**

1. Cloner le repository

```

git clone https://github.com/ton-equipe/schoolpay.git
cd schoolpay

```

2. Configuration Backend (PHP/API)

```

# Copier la configuration
cd backend
cp config/database.example.php config/database.php

# Éditer les credentials MySQL
nano config/database.php

```

```
# Structure attendue:
# DB_HOST=localhost
# DB_NAME=schoolpay_db
# DB_USER=root
# DB_PASS=votre_mot_de_passe
```

3. Installation base de données

```
# Créer la base de données
mysql -u root -p < database/schema.sql
mysql -u root -p < database/seed.sql # Données de test
```

4. Configuration Frontend (React)

```
cd ../frontend
npm install

# Créer le fichier d'environnement
cp .env.example .env

# Éditer l'URL de l'API
nano .env
# REACT_APP_API_URL=http://localhost/schoolpay/backend/api
```

5. Lancement en développement

```
# Terminal 1 - Backend (si serveur PHP intégré)
cd backend
php -S localhost:8000

# Terminal 2 - Frontend
cd frontend
npm start
```

Accès : <http://localhost:3000> (React dev server avec proxy API)

Structure du Projet

```
schoolpay/
├──  backend/           # API PHP
│   ├──  api/             # Points d'entrée REST
│   │   ├── auth.php      # Login/Logout/Refresh
│   │   ├── eleves.php    # CRUD élèves
│   │   ├── paiements.php # Gestion paiements
│   │   ├── frais.php     # Configuration frais
│   │   ├── recus.php     # Génération reçus
│   │   └── stats.php     # Statistiques/dashboard
│   └──  config/
```

```

| | | database.php          # Connexion PDO
| | | jwt.php              # Configuration tokens
| | | └─ cors.php          # Headers CORS
| | └─ models/             # Classes métier
| | | Eleve.php
| | | Paiement.php
| | | Agent.php
| | | └─ Recu.php
| └─ middleware/           # Authentification
| | └─ AuthMiddleware.php
| └─ utils/               # Helpers
| | Response.php          # Format JSON uniforme
| | Validator.php         # Validation données
| | └─ PdfGenerator.php   # Génération reçus
| └─ .htaccess            # Rewrite rules API
|
└─ frontend/              # Application React
  └─ public/
  └─ src/
    └─ components/        # Composants réutilisables
      └─ Layout/          # Header, Sidebar, Footer
      └─ Forms/           # Inputs validés
      └─ Tables/          # DataTables interactifs
      └─ Charts/          # Graphiques Recharts
    └─ pages/             # Vues principales
      └─ Login.jsx
      └─ Dashboard.jsx    # Tableau de bord
      └─ Eleves/          # Gestion élèves
      └─ Paiements/       # Enregistrement paiements
      └─ Recus/           # Historique reçus
      └─ Stats/           # Rapports financiers
    └─ hooks/             # Custom hooks
      └─ useAuth.js        # Gestion authentification
      └─ useApi.js         # Appels API centralisés
      └─ useLocalStorage.js
    └─ context/           # State management
      └─ AuthContext.jsx
      └─ AppContext.jsx
    └─ services/          # API clients
      └─ api.js            # Axios instance
    └─ utils/             # Helpers
      └─ formatters.js     # Format monnaie/dates
      └─ validators.js
    └─ App.jsx
    └─ index.js
  └─ package.json
  └─ tailwind.config.js
|
└─ database/              # Schémas SQL
  └─ schema.sql           # Structure complète
  └─ seed.sql             # Données initiales

```

	└─ migrations/	# Versions futures
	└─ docs/	# Documentation
	└─ MERISE/	# Modèles conceptuels
	└─ API.md	# Documentation endpoints
	└─ DEPLOY.md	# Guide déploiement
	└─ tests/	# Tests automatisés
	└─ backend/	# Tests PHP (PHPUnit)
	└─ frontend/	# Tests React (Jest/Cypress)
	└─ README.md	

🔑 API Endpoints

Authentification

Méthode	Endpoint	Description
POST	/api/auth/login	Connexion agent (matricule + mot de passe)
POST	/api/auth/logout	Déconnexion
POST	/api/auth/refresh	Rafraîchir token JWT

Élèves

Méthode	Endpoint	Description
GET	/api/eleves	Liste avec pagination/filtres
GET	/api/eleves/:id	Détail élève + historique paiements
POST	/api/eleves	Création nouvel élève
PUT	/api/eleves/:id	Modification
DELETE	/api/eleves/:id	Suppression logique

Paievements

Méthode	Endpoint	Description
GET	/api/paiements	Liste transactions
POST	/api/paiements	Enregistrer paiement
GET	/api/paiements/:id/recu	Générer reçu PDF
GET	/api/paiements/impayes	Liste impayés par classe

Statistiques

Méthode	Endpoint	Description
GET	/api/stats/dashboard	KPIs tableau de bord
GET	/api/stats/par-classe	Répartition par classe
GET	/api/stats/journalier	Recettes journalières
GET	/api/stats/export	Export Excel/PDF

Format de réponse uniforme

```
{
  "success": true,
  "data": { ... },
  "message": "Opération réussie",
  "meta": {
    "page": 1,
    "total": 150
  }
}
```

 Sécurité

Mesures implémentées

Couche	Mesure	Implémentation
Authentification	JWT tokens	firebase/php-jwt
Autorisation	Rôles (admin, caissier, directeur)	Middleware PHP
Données	Requêtes préparées PDO	100% des requêtes
Transport	HTTPS (production)	Certificat SSL
Mots de passe	Hachage bcrypt	COST 12
CORS	Origines contrôlées	Headers dynamiques
Injection	Validation stricte	Regex + whitelist
XSS	Échappement output	htmlspecialchars

Variables d'environnement sensibles

```
# backend/.env
JWT_SECRET=votre_cle_secrete_complexe_32_chars
DB_PASSWORD=mot_de_passe_fort
ENCRYPTION_KEY=cle_chiffrement_donnees

# frontend/.env
REACT_APP_API_URL=https://api.schoolpay.cd
REACT_APP_ENV=production
```

Déploiement

Option 1: Hébergement mutualisé (cPanel)

```
# Build production
cd frontend
npm run build

# Upload via FTP/cPanel File Manager
# - backend/ → public_html/api/
# - frontend/build/ → public_html/
# - database/ → Importer via phpMyAdmin
```

Option 2: VPS Cloud (DigitalOcean/Linode)

```
# Docker Compose (recommandé)
docker-compose up -d

# Services:
# - nginx (reverse proxy + frontend)
# - php-fpm (backend API)
# - mysql (base de données)
# - redis (cache sessions)
```

Configuration serveur Apache (.htaccess)

```
# backend/.htaccess
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php [QSA,L]

# Headers sécurité
Header always set X-Frame-Options "SAMEORIGIN"
Header always set X-Content-Type-Options "nosniff"
```

Roadmap

Phase 1 - MVP (Mai 2026)

- Gestion élèves et agents
- Enregistrement paiements cash
- Génération reçus PDF
- Tableau de bord basique
- Intégration mobile money (API opérateurs)

Phase 2 - Scale (Juin-Juillet 2026)

- Multi-école (4 établissements pilotes)
- Portail parents (consultation en ligne)

- Application mobile React Native
- Système de relance automatique (SMS)

Phase 3 - Innovation (2027)

- Paiement en ligne carte bancaire
- Blockchain pour traçabilité
- IA pour prédiction impayés
- Intégration écoles publiques (Ministère)



Documentation complémentaire

-  [Guide MERISE](#)
-  [API Reference](#)
-  [Déploiement Production](#)
-  [Contribution](#)



Contribution

1. Fork le projet
2. Créer une branche (`git checkout -b feature/amelioration`)
3. Commit (`git commit -m 'Ajout fonctionnalité X'`)
4. Push (`git push origin feature/amelioration`)
5. Ouvrir une Pull Request

Standards de code:

- PHP: PSR-12
- React: ESLint Airbnb + Prettier
- Commits: Conventionnelles (feat:, fix:, docs:)



Support & Contact

Équipe SchoolPay - IKAB

-  Email: contact@schoolpay.cd
-  Téléphone: +243 XX XXX XXXX
-  Adresse: Kinshasa, RDC

Rapports de bugs: [GitHub Issues](#)



Licence

Ce projet est sous licence MIT. Voir le fichier [LICENSE](#) pour plus de détails.

Avertissement: Ce système gère des données financières sensibles. Assurez-vous de respecter la législation locale sur la protection des données (RGPD/loi congolaise).

Retour en haut

Développé avec ❤️ par l'équipe SchoolPay - HEC Kinshasa 2024-2025